



RouteZone

Application de prédiction de la gravité des accidents de la route

Rapport E3 – Intégration des modèles et des services d'intelligence artificielle

Meriem ABDELOUAHED

Formation Développeur en Intelligence Artificielle — Simplon × Microsoft

Titre RNCP37827

github.com/Meriem69/Routezone

Sommaire

1. Introduction et compétences couvertes	3
2. Architecture globale du service IA	4
3. API REST IA — FastAPI	6
4. Sécurité, authentification et conformité	8
5. Modèle ML servi (LightGBM)	11
6. Application prototype Streamlit	13
7. Tests unitaires et fonctionnels	15
8. Pipeline CI/CD GitHub Actions	16
9. Monitoring temps réel	17
10. MLflow tracking	19
11. Conclusion	20

1. Introduction et compétences couvertes

1.1 Présentation rapide de RouteZone

RouteZone est une application de prédiction de la gravité des accidents routiers (Grave / Pas grave), basée sur les données BAAC 2022-2024 publiées en open data par le Ministère de l'Intérieur. Le modèle retenu est LightGBM V3 OSRM, enrichi des temps réels d'intervention des secours (pompiers et SAU) calculés via le moteur de routage OSRM.

Le rapport E1 a couvert la donnée (collecte, nettoyage, base PostgreSQL, API data). Le rapport E2 a détaillé la veille technologique et le choix du modèle. Ce rapport E3 présente la suite logique : la mise en service complète du modèle, transformé d'un fichier .pkl en API REST sécurisée, conteneurisée, testée, monitorée et utilisable via une interface Streamlit.

1.2 Problématique de l'épreuve E3

Comment ce modèle entraîné est-il devenu un service réel ? Le modèle LightGBM ne tourne pas seul. Il s'intègre dans une architecture complète :

- Une API REST FastAPI sécurisée (16 endpoints, authentification API Key + JWT, hashage bcrypt)
- Une base PostgreSQL pour les utilisateurs et l'historique des prédictions
- Un moteur OSRM pour les calculs de temps d'intervention en temps réel
- 35 tests automatisés (dont 1 test de non-régression Recall) exécutés à chaque push via GitHub Actions
- Un monitoring complet (Prometheus + Grafana + Python Logging)
- Une application Streamlit pour l'interface utilisateur
- MLflow pour la traçabilité du cycle de vie du modèle
- Le tout conteneurisé via Docker et orchestré par docker-compose

1.3 Compétences couvertes

Ce rapport adresse explicitement les compétences C9 à C13 du référentiel RNCP37827 :

Code	Compétence	Section du rapport
C9	Développer une API exposant le modèle IA	Sections 2, 3
C10	Intégrer l'API du modèle dans l'application	Sections 5, 6
C11	Monitorer le modèle (approche MLOps)	Section 9,10
C12	Programmer les tests automatisés du modèle	Section 7
C13	Créer la chaîne de livraison continue du modèle	Section 8

2. Architecture globale du service IA

2.1 Vue d'ensemble

Le projet RouteZone tourne sur 5 containers Docker orchestrés par docker-compose, qui communiquent entre eux via un réseau Docker dédié :

- routezone_db (PostgreSQL 16) — port 5432
- routezone_osrm (moteur de routage Open Source Routing Machine) — port 5000
- routezone_api (FastAPI unifiée) — port 8001
- routezone_prometheus (collecte des métriques) — port 9090
- routezone_grafana (visualisation des dashboards) — port 3000

Le frontend Streamlit tourne en dehors des containers, en local sur le port 8501. Cette architecture orientée services garantit la séparation des responsabilités, la résilience (si un service tombe, les autres continuent), et la reproductibilité totale via une seule commande : `docker-compose up -d`.

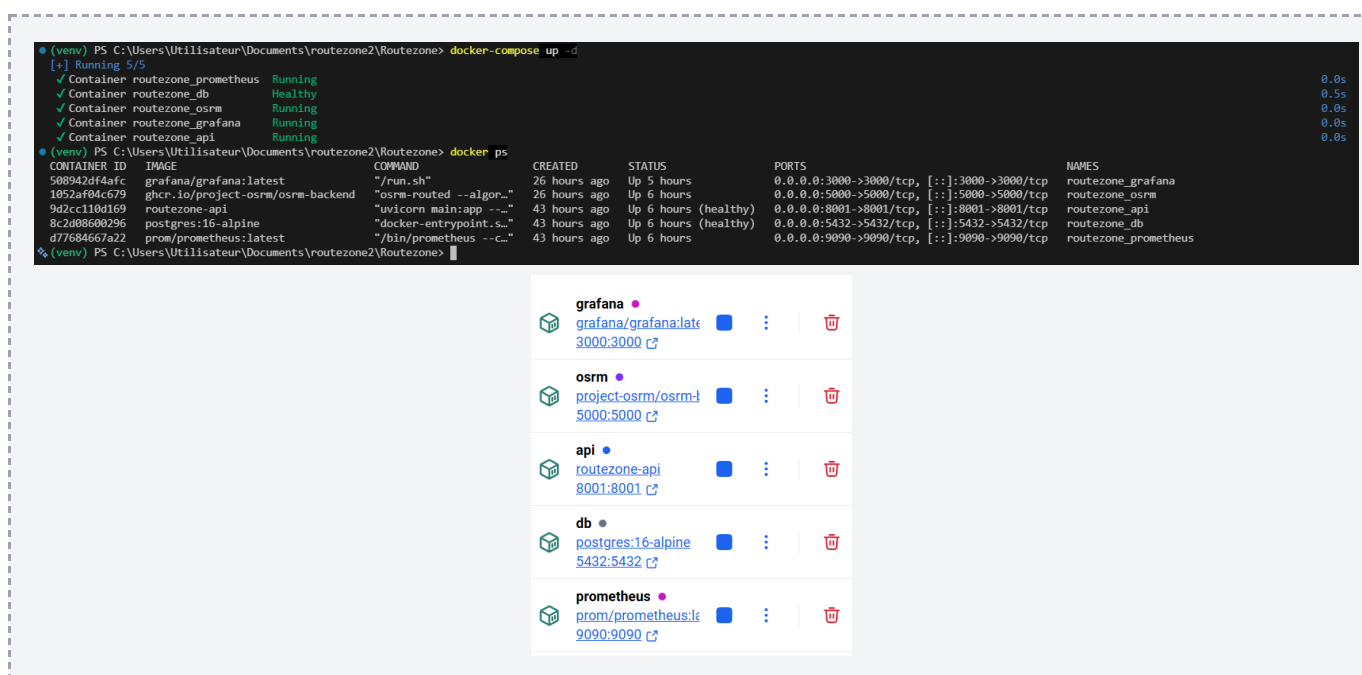


Figure 1 — Architecture du service IA RouteZone (5 containers Docker)

2.2 Flux de données pour une prédiction

Le cycle de vie d'une prédiction suit 7 étapes :

- L'utilisateur saisit un accident dans Streamlit (type de route, âge, luminosité, météo, coordonnées GPS, etc.)
- Streamlit appelle OSRM pour calculer les temps d'intervention réels (caserne -> accident -> SAU) et détecter les zones blanches
- Streamlit envoie une requête POST `/predict` à l'API, avec le JWT dans l'en-tête Authorization
- L'API vérifie le JWT (module `security.py`) et valide les données avec Pydantic
- L'API utilise le modèle LightGBM déjà chargé en mémoire pour effectuer la prédiction
- L'API enregistre le résultat dans PostgreSQL avec l'identifiant de l'utilisateur connecté
- L'API retourne la réponse à Streamlit (label + probabilité) et incrémente les compteurs Prometheus

2.3 Justification des choix d'architecture

Pourquoi Docker : fiabilité, portabilité et reproductibilité. Une seule commande docker-compose up suffit à lancer toute la stack.

Pourquoi une API unifiée plutôt que 2 séparées : à l'origine j'avais 2 API distinctes (Data et IA). Je les ai fusionnées pour gagner en simplicité opérationnelle (un seul container, une seule Swagger, authentification mutualisée). Une fonction de routes_ia.py peut désormais appeler une fonction de routes_data.py directement, sans passer par le réseau.

Pourquoi PostgreSQL plutôt que SQLite : justifié dans le rapport E1. Meilleure gestion de la concurrence, support des types géospatiaux pour la carte Streamlit, conformité ACID.

3. API REST IA — FastAPI

3.1 Pourquoi FastAPI

DÉFINITION — **FastAPI** framework Python moderne basé sur Starlette et Pydantic, conçu pour la rapidité d'exécution et la documentation automatique. Créé en 2018, c'est l'un des frameworks Python les plus rapides du marché.

FastAPI a été retenu pour quatre raisons :

- Validation automatique des entrées via Pydantic et les type hints Python
- Documentation Swagger générée automatiquement, accessible sur /docs
- Performances asynchrones natives via Starlette
- Sécurité par défaut (validation stricte, support JWT natif)

3.2 Architecture interne de l'API

L'API est structurée selon le principe de séparation des responsabilités :

- 1 point d'entrée : main.py qui crée l'instance FastAPI et monte les routers
- 3 routers : routes_auth.py, routes_data.py, routes_ia.py
- 4 modules transverses : security.py (JWT/bcrypt), db.py (SQLAlchemy), logger.py (logs), metrics.py (Prometheus)

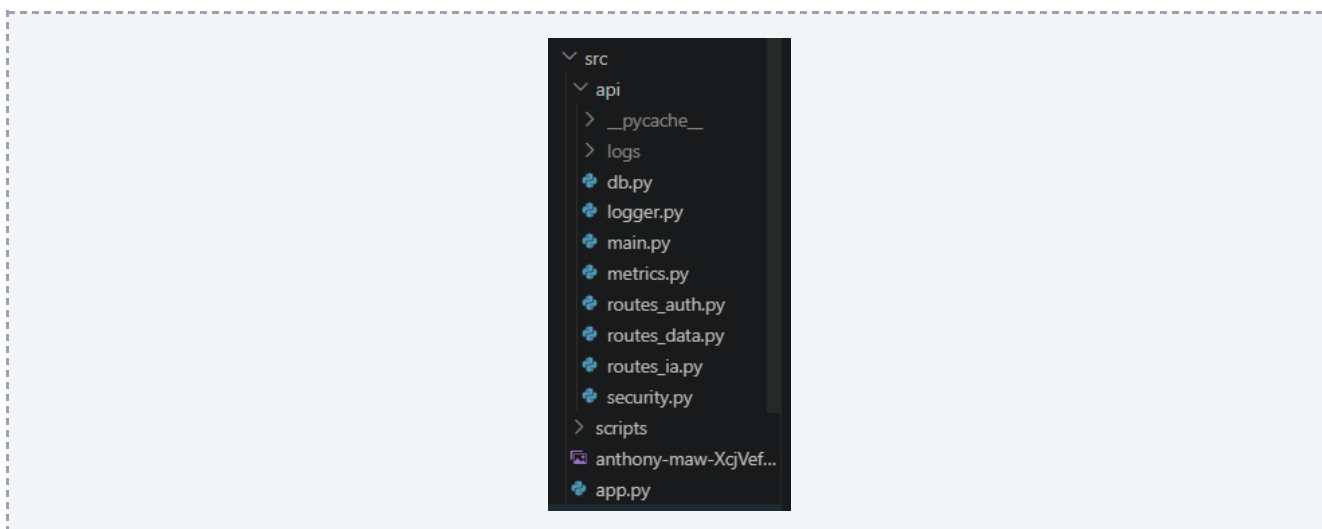


Figure 2 — Architecture interne de l'API RouteZone

3.3 16 endpoints documentés via Swagger UI

L'API expose 16 endpoints uniques, organisés en 6 catégories dans Swagger UI : Auth, Données BAAC, Intelligence Artificielle, RGPD, Health, Monitoring. Les 2 endpoints RGPD apparaissent deux fois dans la documentation Swagger car ils sont taggués à la fois dans 'Intelligence Artificielle' et 'RGPD', ce qui facilite leur découverte selon l'angle d'utilisation. FastAPI génère automatiquement une page Swagger UI accessible sur <http://localhost:8001/docs>.

DÉFINITION — Swagger UI interface web qui transforme une spécification OpenAPI en documentation cliquable et testable. Permet de tester chaque endpoint directement depuis le navigateur, sans avoir besoin d'installer Postman.

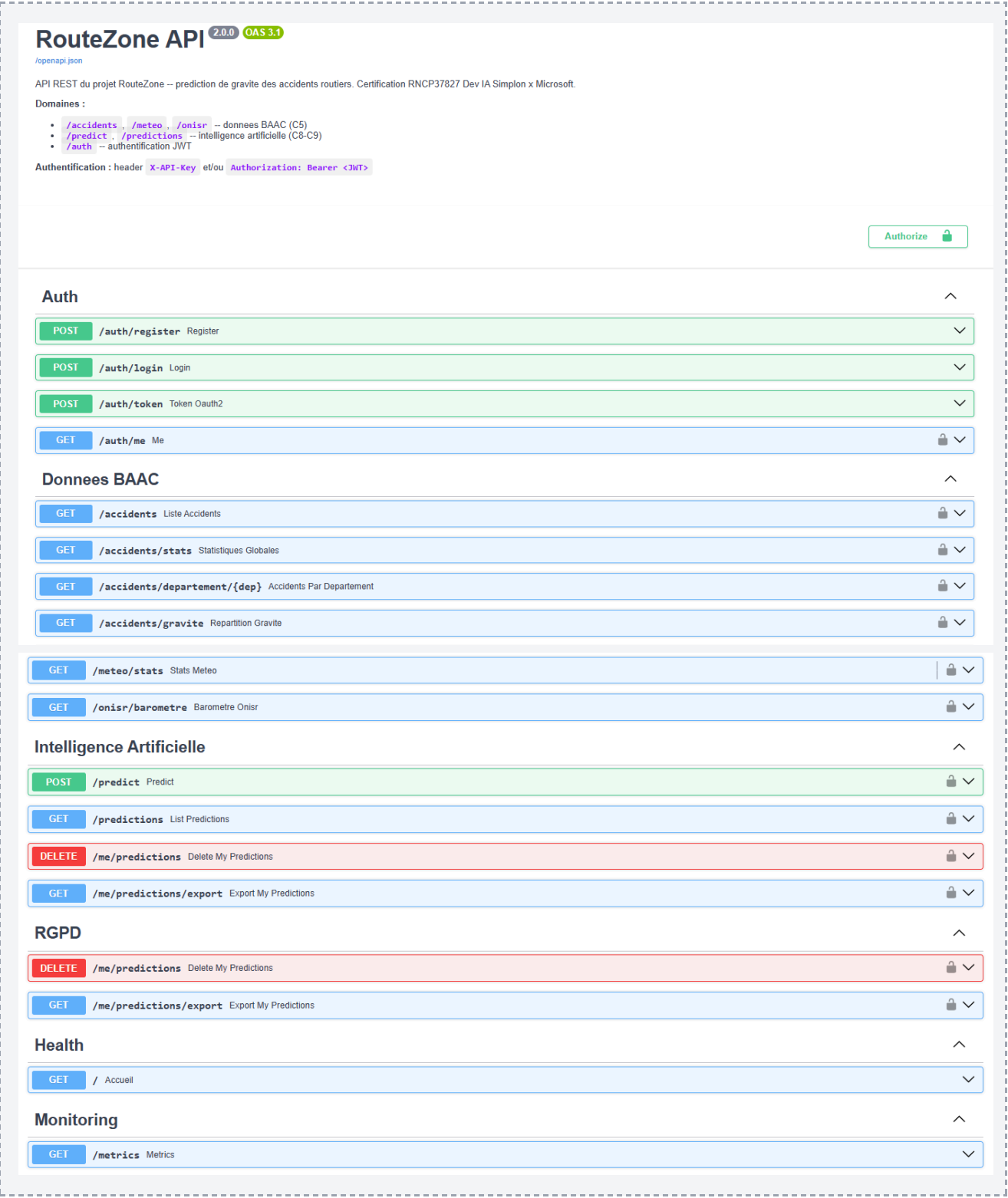


Figure 3 — Documentation Swagger interactive des 16 endpoints

3.4 Validation des entrées avec Pydantic

Pydantic valide automatiquement chaque champ envoyé à l'API. Si les contraintes ne sont pas respectées (par exemple un âge à 999), l'API renvoie une erreur HTTP 422 sans même appeler le modèle. Protection efficace contre les données aberrantes.

Exemple de validation sur AccidentInput : âge entre 0 et 120, heure entre 0 et 23, mois entre 1 et 12, latitude entre -90 et 90, vitesse maximale autorisée entre 0 et 200 km/h. Tous ces contrôles sont déclarés en quelques lignes et FastAPI les applique automatiquement à chaque requête.

<pre>72 class AccidentInput(BaseModel): 73 lum: int 74 agg: int 75 int_: int 76 atm: int 77 col: int 78 catr: int 79 circ: int 80 vosp: int 81 prof: int 82 plan: int 83 surf: int 84 infra: int 85 situ: int 86 vma: int = Field(..., ge=0, le=200) 87 catu: int 88 sexe: int 89 trajet: int 90 secu1: int 91 catv: int 92 age: int = Field(..., ge=0, le=120) 93 heure: int = Field(..., ge=0, le=23) 94 mois: int = Field(..., ge=1, le=12) 95 temperature: float = Field(..., ge=-30, le=50) 96 precipitation: float = Field(..., ge=0) 97 windspeed: float = Field(..., ge=0, le=300) 98 jour: int = 15 99 lat: float = Field(None, ge=-90, le=90) 100 lon: float = Field(None, ge=-180, le=180)</pre>	Lexique Pydantic ge — Greater or Equal → $\geq$ (supérieur ou égal) le — Less or Equal → $\leq$ (inférieur ou égal) gt — Greater Than → $>$ (strictement supérieur) lt — Less Than → $<$ (strictement inférieur) Field(...) — Le ... rend le champ obligatoire. Erreur 422 si manquant. Field(None, ...) — Le None rend le champ optionnel. Valeur par défaut = None.
--	--

Figure 2 — Class AccidentInput + Lexique

3.5 Gestion structurée des erreurs

L'API renvoie des codes HTTP cohérents avec leur signification métier :

- 200 — Prédiction réussie (cas nominal)
- 401 — Token JWT invalide ou expiré (plus de 24h)
- 403 — Authentification manquante (clé API absente)
- 404 — Ressource non trouvée (département inexistant par exemple)
- 409 — Conflit (email déjà utilisé à l'inscription)
- 422 — Données invalides (âge à 224 ans, etc.)
- 500 — Erreur serveur (base de données inaccessible, etc.)

4. Sécurité, authentification et conformité

4.1 Double système d'authentification

Authentification par clé API (machines)

Sert quand un autre logiciel veut utiliser mon API (pas un humain). Le client envoie un header X-API-Key. La comparaison se fait avec `hmac.compare_digest` qui empêche les attaques par mesure de temps : l'opérateur `==` classique s'arrête dès qu'il trouve une différence, ce qui permet à un attaquant de deviner la clé caractère par caractère en mesurant les temps de réponse. `hmac.compare_digest` met toujours le même temps, quelle que soit l'entrée.

Authentification par JWT (utilisateurs humains)

DÉFINITION — JWT (JSON Web Token) chaîne signée cryptographiquement par l'API avec une clé secrète. Si quelqu'un modifie le contenu du token (par exemple change le `user_id` pour devenir admin), la signature ne correspond plus et l'API rejette le token.

Le protocole HTTP est sans mémoire : chaque requête est indépendante. À chaque requête, l'utilisateur renvoie son JWT dans le header `Authorization: Bearer`. L'API vérifie la signature sans consulter la BDD, ce qui rend le JWT plus performant que les sessions classiques. Le token expire au bout de 24h.

4.2 Hashage bcrypt des mots de passe

DÉFINITION — bcrypt algorithme de hashage à sens unique conçu spécifiquement pour les mots de passe. Lent volontairement (tester des milliards de combinaisons prend des années, contre quelques heures avec MD5), et ajoute un sel automatique (deux utilisateurs avec le même mot de passe auront deux hashs différents).

Les mots de passe ne sont jamais stockés en clair en base de données. Sans cette précaution, en cas de fuite, tous les mots de passe seraient publics. À l'inscription, `passlib` avec `bcrypt` transforme `motdepasse123` en une chaîne du type `$2b$12$EixZaYVK1fsbw1ZfbX3OXe....`. Cette transformation est à sens unique : on peut hasher mais pas revenir au mot de passe d'origine.

4.3 Conformité RGPD

DÉFINITION — RGPD Règlement Général sur la Protection des Données. Obligatoire pour toute application traitant des données personnelles dans l'Union Européenne. Repose sur 6 principes : minimisation, transparence, droits des utilisateurs, durée de conservation, sécurité, démarche continue.

Dans RouteZone, deux endpoints dédiés ont été implémentés, tous deux protégés par JWT (un utilisateur ne peut accéder qu'à ses propres données) :

- DELETE /me/predictions — Article 17 RGPD (droit à l'effacement). L'utilisateur peut supprimer toutes ses prédictions en une requête
- GET /me/predictions/export — Article 20 RGPD (droit à la portabilité). L'utilisateur peut récupérer ses données au format JSON

RGPD

DELETE /me/predictions Delete My Predictions

GET /me/predictions/export Export My Predictions

4.4 Sécurité OWASP

DÉFINITION — OWASP Open Web Application Security Project. Fondation internationale qui publie tous les 4 ans un classement des 10 vulnérabilités les plus critiques pour les applications web : le Top 10 OWASP.

Mesures OWASP adressées dans RouteZone :

Vulnérabilité OWASP	Mesure mise en place
A01 — Contrôles d'accès défaillants	Filtre SQL WHERE user_id = :uid sur tous les endpoints /me/*
A02 — Défaillances cryptographiques	Mots de passe hashés avec bcrypt, JWT signé avec HS256
A03 — Injection	SQLAlchemy ORM avec paramètres bindés (immune aux injections SQL)
A07 — Identification de mauvaise qualité	JWT avec expiration 24h, hmac.compare_digest, bcrypt
A09 — Carence de journalisation	Toutes les actions sensibles tracées dans logs/api.log

4.5 Limites de sécurité assumées

- Pas de rate limiting : un utilisateur peut envoyer un nombre illimité de requêtes. Un attaquant pourrait tenter des milliers de mots de passe par minute. Amélioration prévue : slowapi
- Pas de HTTPS en local : l'API tourne en HTTP. En production il faudrait un reverse proxy (nginx, Traefik) avec certificat SSL/TLS

5. Modèle ML servi (LightGBM)

5.1 Rappel du modèle

Le modèle retenu est LightGBM V3 OSRM brut (sans calibration). Performances sur le test 2024 :

- - Recall GRAVE : 0.7643
- - Precision GRAVE : 0.4166
- - F1 macro : 0.6956
- - AUC-ROC : 0.8558
- - Accuracy : 0.7760

```

TRAIN : recall=0.8471 prec=0.4595 f1=0.7304 auc=0.8957
VAL   : recall=0.7984 prec=0.4338 f1=0.7081 auc=0.8693
TEST  : recall=0.7643 prec=0.4166 f1=0.6956 auc=0.8558

Overfit check : recall_train (0.8471) - recall_test (0.7643) = +0.0828
-> léger overfit

=== V3 OSRM - Test 2024 ===
      precision    recall  f1-score   support

   Pas grave      0.94      0.78      0.85    123037
    Grave        0.42      0.76      0.54     25469

 accuracy
macro avg      0.68      0.77      0.70    148506
weighted avg    0.85      0.78      0.80    148506

```

Figure 3 — Performances V3 OSRM sur le test 2024. Le split temporel garantit qu'aucun accident 2024 n'a été vu pendant l'entraînement (réalisé sur 2022-2023). L'écart Recall TRAIN/TEST de seulement +0.0828 confirme une bonne capacité de généralisation.

Pour le détail du choix du modèle et la comparaison V1 / V2 / V3, voir le rapport E2 (section 6.5).

5.2 Chargement du modèle au démarrage

Le modèle est chargé une seule fois au démarrage de l'API, pas à chaque requête, ce qui évite la latence d'I/O répétée.

Trois artefacts sont chargés :

- `best_model_v3_osrm.pkl` : le modèle entraîné (sérialisation joblib)
- `features_v3_osrm.pkl` : la liste des 34 features dans l'ordre exact attendu
- `class_mapping.pkl` : correspondance sorties modèle (0/1) vers labels lisibles (Pas grave / Grave)

DÉFINITION — joblib bibliothèque Python spécialisée pour la sérialisation d'objets numériques volumineux. Plus rapide que pickle standard sur les gros modèles ML.

5.3 Stratégie multi-versions et fallback automatique

Le code de chargement implémente un fallback automatique en cas de fichier manquant : priorité V3 OSRM, sinon V2 Haversine, sinon V1 baseline. Cette stratégie garantit que l'API ne tombe jamais en panne à cause d'un artefact manquant, et permet la traçabilité complète de l'évolution V1 -> V2 -> V3.

5.4 Pipeline de prédiction

Le pipeline /predict suit 7 étapes :

- Réception des données utilisateur (validées par Pydantic)
- Calcul des features dérivées (créneau horaire, tranche d'âge, jour de la semaine)
- Construction du DataFrame dans l'ordre exact des features attendues
- Conversion des features catégorielles en type category (format LightGBM)
- Prédiction avec `model.predict_proba(X)`
- Application du seuil 0.5 : probabilité ≥ 0.5 -> Grave, sinon Pas grave
- Retour de la réponse JSON (label + probabilité + temps d'intervention si fournis)

Intelligence Artificielle

POST

/predict Predict

Predit la gravite d'un accident. Sauvegarde dans l'historique si JWT fourni.

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value

Schema

```
{
  "infra": 0,
  "situ": 1,
  "vma": 130,
  "catu": 1,
  "sexe": 1,
  "trajet": 5,
  "secut": 1,
  "catv": 33,
  "age": 25,
  "heure": 23,
  "mois": 11,
  "temperature": 8.5,
  "precipitation": 3.2,
  "windspeed": 25,
  "jour": 15,
  "lat": 45.7640,
  "lon": 4.8357,
  "nearest_pompiers_min": 12.5,
  "nearest_sau_min": 18.3
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8001/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "lum": 3,
    "agg": 2,
    "int_": 1,
    "ata": 2,
    "col": 3,
    "catr": 1,
    "circ": 2,
    "vosp": 0,
    "prof": 1,
    "plan": 1,
    "surf": 2,
    "infra": 0,
    "situ": 1,
    "vma": 130,
    "catu": 1,
    "sexe": 1,
    "trajet": 5,
    "secut": 1,
    "catv": 33,
    "age": 25,
    "heure": 23,
    "mois": 11,
    "temperature": 8.5,
    "precipitation": 3.2,
    "windspeed": 25,
    "jour": 15,
    "lat": 45.7640,
    "lon": 4.8357,
    "nearest_pompiers_min": 12.5,
    "nearest_sau_min": 18.3
  }'
```

Request URL

http://localhost:8001/predict

Server response

Code

Details

Undocumented

Failed to fetch.
Possible Reasons:

- CORS
- Network Failure
- URL scheme must be "http" or "https" for CORS request.

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value

Schema

```
"string"
```

Figure 4 — Test du endpoint /predict via Swagger UI

5.5 Décision technique : calibrator désactivé

DÉFINITION — Calibrator isotonique méthode de post-traitement statistique qui ajuste les probabilités d'un modèle pour qu'elles correspondent aux fréquences réelles observées. Apprend une fonction monotone croissante par morceaux.

Le calibrator isotonique scikit-learn est censé rendre les probabilités plus interprétables. Dans mon cas, je ne peux pas l'utiliser car il dégrade le Recall sur la classe Grave (passe de 76 % à 33 %). Concrètement, si le calibrator restait activé, il classerait à tort deux tiers des accidents graves comme Pas grave, ce qui n'est pas tolérable pour un système d'aide aux services de secours.

Je l'ai donc désactivé dans routes_ia.py (lignes 32-38). J'ai conservé le fichier .pkl du calibrator pour la traçabilité. La démarche d'investigation est entièrement documentée dans docs/incidents/incident_calibrator.md.

```
=== CALIBRATOR V3 OSRM (isotonic) - Test 2024 ===
              precision    recall  f1-score   support

   Pas grave      0.87      0.96      0.92    123037
    Grave        0.65      0.33      0.44     25469

 accuracy              0.85    148506
 macro avg           0.76      0.65      0.68    148506
 weighted avg        0.84      0.85      0.83    148506

=== Comparaison brut vs calibre ===
              Modele brut V3  Calibrator isotonic  delta
recall_grave              0.7643              0.3299 -0.4343
precision_grave           0.4166              0.6504  0.2338
f1_macro                  0.6956              0.6772 -0.0185
auc_roc                   0.8558              0.8458 -0.0099
accuracy                  0.7760              0.8547  0.0787
```

Figure 5 — Metrics Calibrator Isotonique

5.6 Performances en production

- Temps de prédiction moyen : moins de 100 ms
- Le test test_predict_temps_reponse_raisonnable valide une réponse en moins de 2 secondes
- Le test test_predict_reproductibilite garantit qu'une même entrée produit toujours la même prédiction

6. Application prototype Streamlit

6.1 Présentation de Streamlit

DÉFINITION — Streamlit bibliothèque Python open source qui permet de créer des applications web data/ML en quelques lignes de code. Pas besoin de HTML/CSS/JS, idéal pour les prototypes rapides.

6.2 Utilisateurs cibles

- Services de secours : pour prioriser les interventions sur les cas potentiellement graves
- Compagnies d'assurance : pour évaluer la gravité probable d'un sinistre
- Collectivités : pour identifier les zones à risque
- Étudiants et chercheurs : pour analyser les données BAAC enrichies

6.3 Parcours utilisateur en 4 pages

Page 1 — Formulaire de prédiction

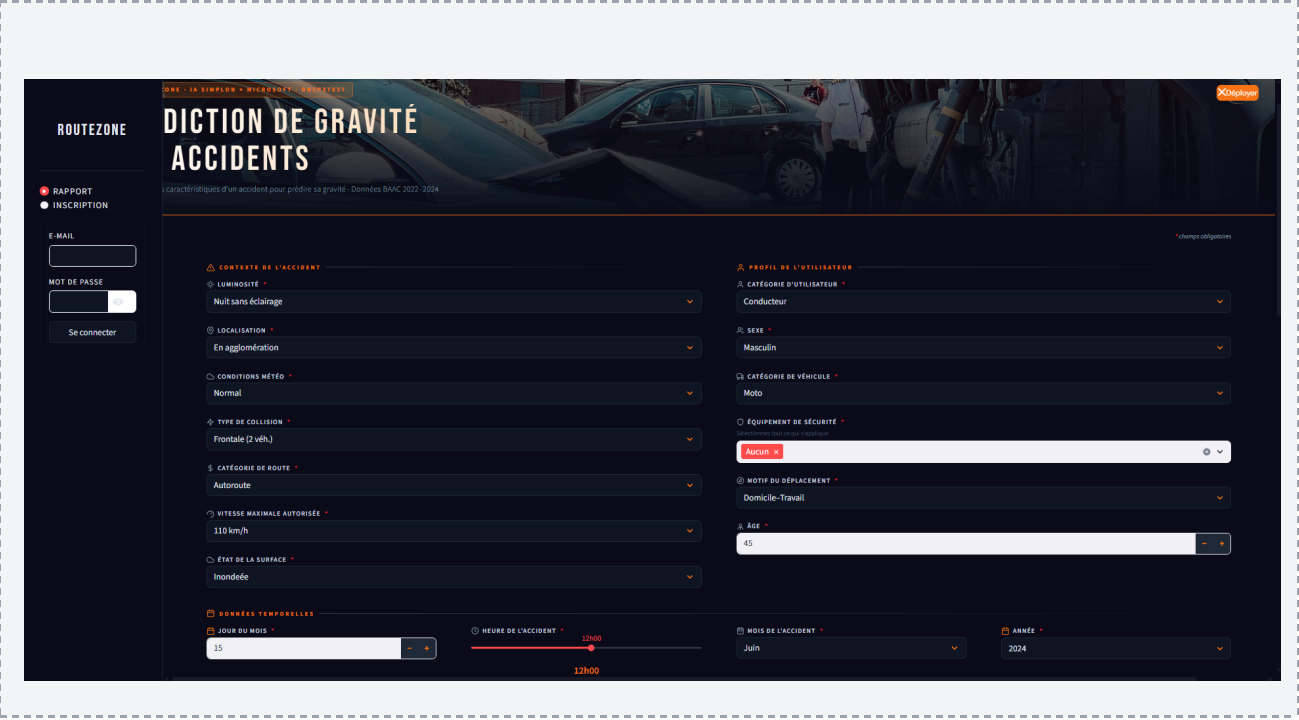


Figure 6 — Page de prédiction Streamlit

Page 2 — Résultat avec carte interactive

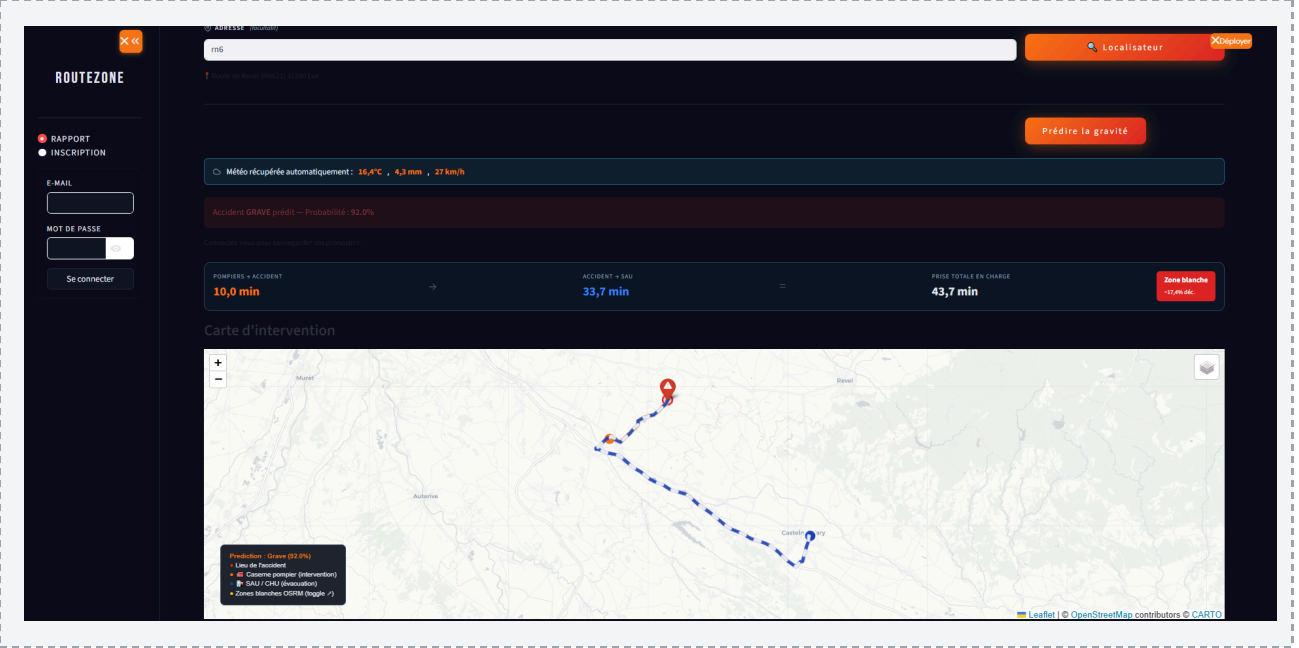


Figure 7 — Résultat d'une prédiction avec carte interactive

Page 3 — Historique des prédictions

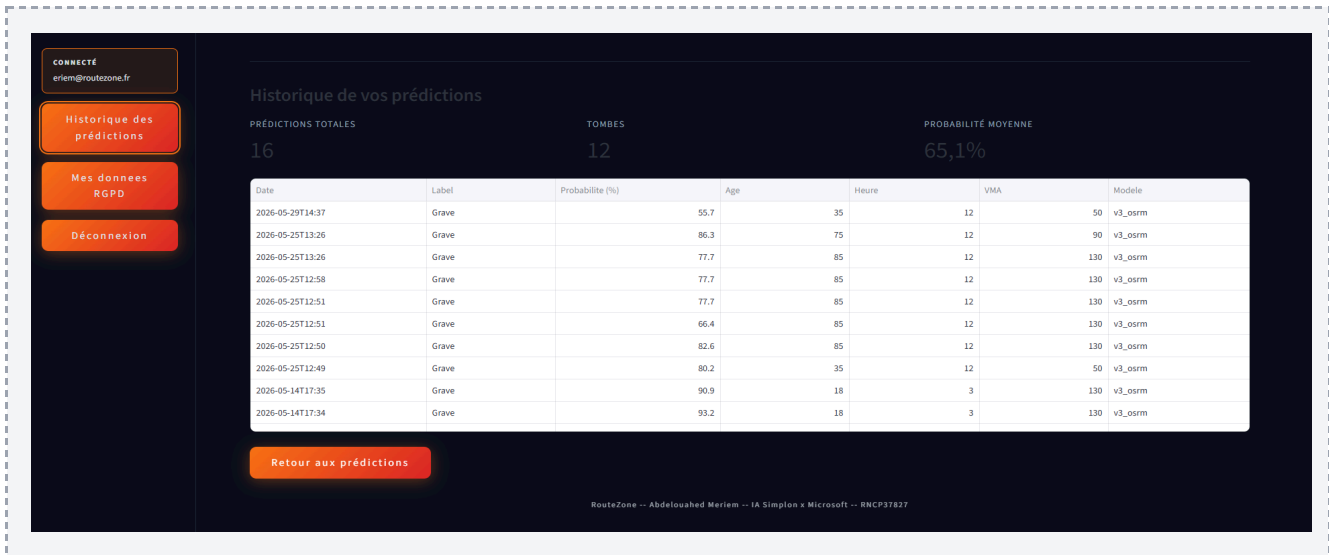


Figure 8 — Page Historique des prédictions

Page 4 — Gestion RGPD

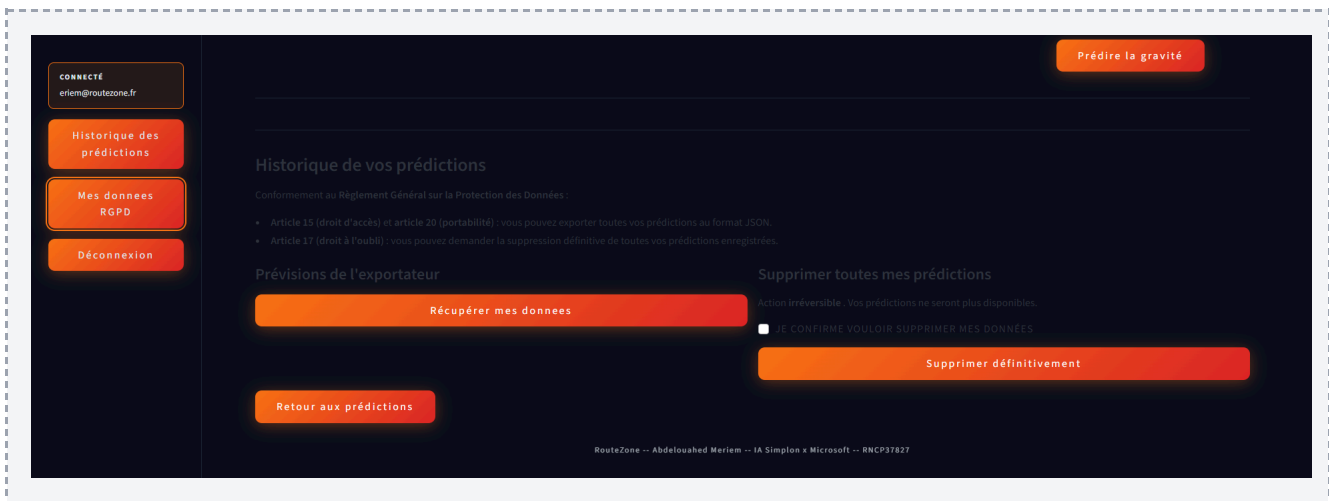


Figure 9 — Page Mes données RGPD

6.4 Communication entre Streamlit et l'API

Streamlit utilise la bibliothèque Python requests pour envoyer des requêtes HTTP à l'API. Le JWT est stocké dans `st.session_state` après le login. À chaque appel API, le JWT est envoyé dans le header `Authorization: Bearer`. Si le token est expiré (24h), l'API renvoie 401 et Streamlit demande à l'utilisateur de se reconnecter.

6.5 Design et identité visuelle

- Palette : Orange #F97316 (signature RouteZone), Bleu nuit #0A0E1A (fond), Gris foncé #111827 (cartes)
- Accents : rouge #EF4444 et vert #22C55E (feu tricolore pour le résultat)
- Typographies : Bebas Neue (titres) et DM Sans (corps), via Google Fonts
- Mode sombre par défaut : réduit la fatigue visuelle pour les services de secours en horaires variés
- Icônes SVG plutôt qu'emojis : rendu identique sur tous les navigateurs

6.6 Limites assumées

- Pas de tests automatisés sur Streamlit : compensé par 35 tests pytest sur l'API
- Pas de fonction mot de passe oublié : amélioration prévue avec email + lien de réinitialisation
- Personnalisation visuelle limitée par les composants natifs Streamlit

7. Tests unitaires et fonctionnels

7.1 Pourquoi tester

- Détecter les régressions : une modification ne doit pas casser l'existant
- Garantir la fiabilité : un bug coûte cher sur un outil d'aide aux secours
- Déployer en confiance : tests verts = push serein
- Documentation vivante : un test bien nommé décrit le comportement attendu

7.2 Framework choisi : pytest

DÉFINITION — **pytest** framework de test le plus utilisé dans l'écosystème Python. Syntaxe simple (fonctions `test_*`), bonne intégration FastAPI via TestClient, exécution rapide.

7.3 Organisation des 35 tests

- Authentification (4 tests) : inscription, login, refresh, logout
- Endpoints data (8 tests) : accidents, stats, départements, gravité, météo, ONISR
- Endpoints IA (6 tests) : /predict, /predictions, /me/predictions, RGPD
- Smoke tests modèle (5 tests) : reproductibilité, temps de réponse, classes valides
- Bornes Pydantic (5 tests) : âge négatif, heure > 23, latitude > 90, etc.
- Cohérence métier (5 tests, point fort du projet) : voir détail section 7.4
- Health check (1 test) : route /
- Test de non-régression Recall (1 test) : seuil minimal de performance du modèle (test_recall_minimal.py)

7.4 Focus sur les tests de cohérence métier

5 tests spécifiques vérifient que le modèle a appris la logique métier, pas juste des corrélations statistiques :

- test_frontale_plus_grave_que_arriere
- test_aucun_equipement_plus_grave_que_ceinture
- test_nuit_plus_grave_que_jour
- test_moto_plus_grave_que_voiture
- test_vma_haute_plus_grave_que_vma_basse

Si l'un échoue, c'est que le modèle a perdu une logique fondamentale, à investiguer avant tout déploiement.

7.5 Résultats des tests

```
PS C:\Users\Utilisateur\Documents\routezone2\Routezone> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\Utilisateur\Documents\routezone2\Routezone\venv\Scripts\Activate.ps1)
PS C:\Users\Utilisateur\Documents\routezone2\Routezone> pytest
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\Utilisateur\Documents\routezone2\Routezone
plugins: anyio-4.13.0
collected 35 items

tests\test_api.py .....
tests\test_business_logic.py .....
tests\test_recall_minimal.py .

===== warnings summary =====
venv\lib\site-packages\starlette\formparsers.py:12
  C:\Users\Utilisateur\Documents\routezone2\Routezone\venv\lib\site-packages\starlette\formparsers.py:12: PendingDeprecationWarning: Please use 'import python_multipart' instead.
    import multipart

venv\lib\site-packages\httpx\_client.py:690
venv\lib\site-packages\httpx\_client.py:690
venv\lib\site-packages\httpx\_client.py:690
  C:\Users\Utilisateur\Documents\routezone2\Routezone\venv\lib\site-packages\httpx\_client.py:690: DeprecationWarning: The 'app' shortcut is now deprecated. Use the explicit style 'transport=WSGITransport(app=...)' instead.
    warnings.warn(message, DeprecationWarning)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 35 passed, 4 warnings in 12.76s =====
```

Figure 10 — pytest local : 35 tests collectés, 100% verts en 12 secondes

7.5 Test de non-régression du Recall (pratique MLOps)

Le fichier `test_recall_minimal.py` contient un test supplémentaire qui agit comme un filet de sécurité Machine Learning. À la différence des autres tests qui valident le comportement du code, ce test valide directement les performances du modèle en production.

Concrètement, ce test vérifie que le Recall GRAVE du modèle V3 OSRM reste supérieur à un seuil minimal défini (par exemple 0.70). Si demain je réentraîne le modèle et que les performances chutent en dessous de ce seuil, le test échoue automatiquement et empêche tout déploiement.

C'est une bonne pratique MLOps : on ne teste pas seulement le code, on teste aussi la qualité des prédictions. Cette approche garantit qu'aucun modèle dégradé ne peut atteindre la production, même par mégarde.

8. Pipeline CI/CD GitHub Actions

8.1 Qu'est-ce que la CI/CD

DÉFINITION — CI/CD Continuous Integration / Continuous Delivery. Automatisation des phases de test, intégration et déploiement. Dans RouteZone, la CI est implémentée (tests automatiques à chaque push). Le CD (déploiement automatique) est une amélioration prévue.

8.2 Outil choisi : GitHub Actions

- Gratuit pour les dépôts publics et comptes personnels
- Intégré nativement à GitHub où vit mon code
- Configuration simple en YAML (`.github/workflows/tests.yml`)
- Catalogue d'actions prêtes à l'emploi
- Standard du marché en 2026 (Jenkins demande un serveur, GitLab CI réservé à GitLab)

8.3 Le workflow `tests.yml`

À chaque push sur master, GitHub Actions exécute automatiquement la chaîne suivante :

- Checkout du code depuis le dépôt
- Setup Python 3.11 sur une VM Ubuntu vierge (jetable, créée puis détruite)
- Installation des dépendances depuis `requirements.txt` (avec cache pip)
- Démarrage d'un service PostgreSQL 16 (même version qu'en local pour conditions identiques)
- Import des données BAAC dans la BDD via `bdd/import_data.py`
- Exécution de `pytest tests/ -v --tb=short`
- Notification du résultat (vert ou rouge) dans l'interface GitHub

8.4 Résultats et historique

- 21 workflow runs réalisés au total
- 18 verts (succès), 3 rouges (échecs corrigés en début de projet)
- Durée moyenne : 2 à 3 minutes par run

Les 3 échecs initiaux correspondent à des moments où mon code n'était pas encore stabilisé. Ces erreurs ont été précieuses : prévenue immédiatement par GitHub, j'ai pu corriger avant que le bug ne se propage.

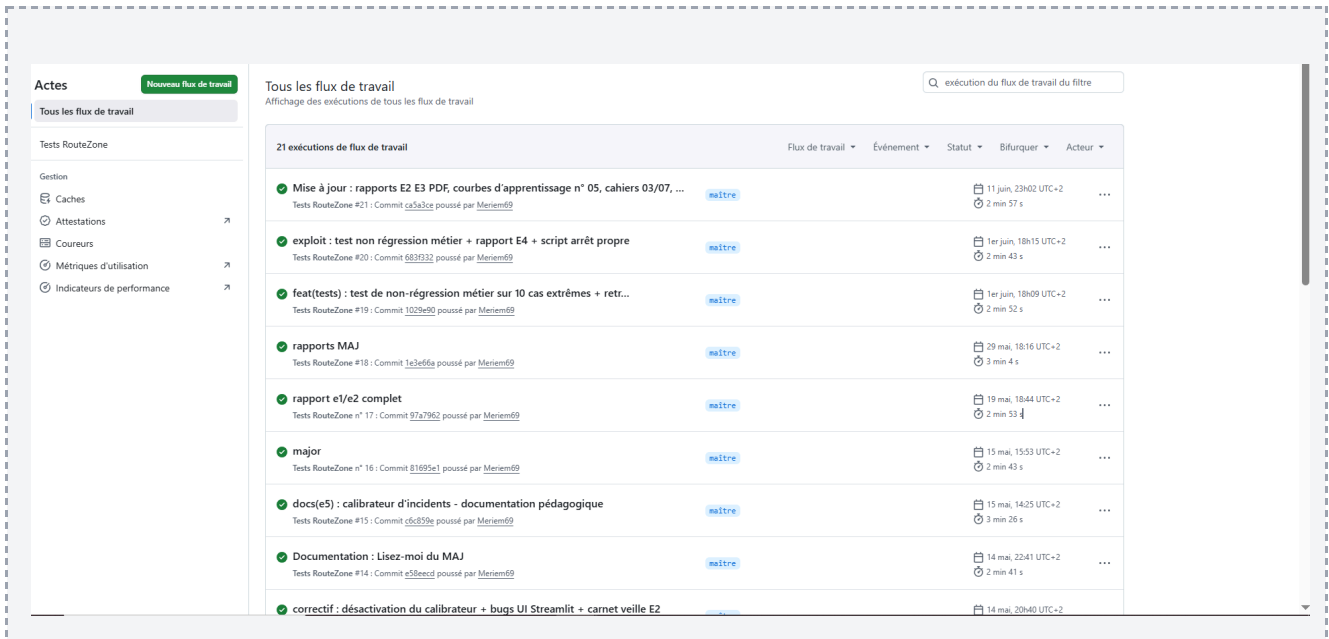


Figure 11 — Historique des workflow runs GitHub Actions (21 runs)

8.5 Du CI au CD : packaging et livraison de l'image

La chaîne ne se limite plus aux tests : elle automatise désormais le packaging et la livraison de l'application. Le workflow comporte deux jobs enchaînés. Le job **test** exécute les 35 tests sur un service PostgreSQL 16. Le job **build-and-push**, qui ne se déclenche que si les tests sont verts (**needs: test**), construit l'image Docker à partir de `docker/Dockerfile.api` et la publie sur le registre de conteneurs de GitHub (GHCR), sous le tag `ghcr.io/meriem69/routezone:latest`.

Concrètement, à chaque push sur master: si un test échoue, la livraison est bloquée ; si tout est vert, une image Docker à jour est automatiquement construite et publiée, prête à être déployée sur n'importe quel serveur. Le packaging (build de conteneur) et la livraison de l'artefact sont donc entièrement automatisés.

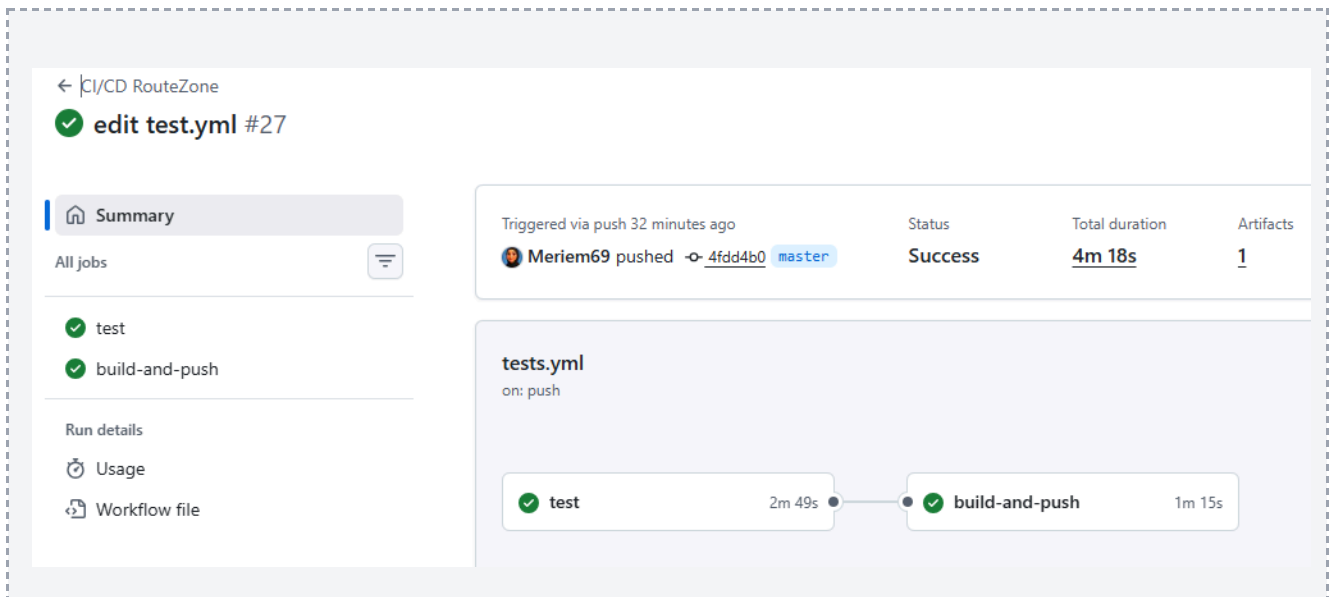


Figure 12 — Run CI/CD : tests verts puis build-and-push enchaînés

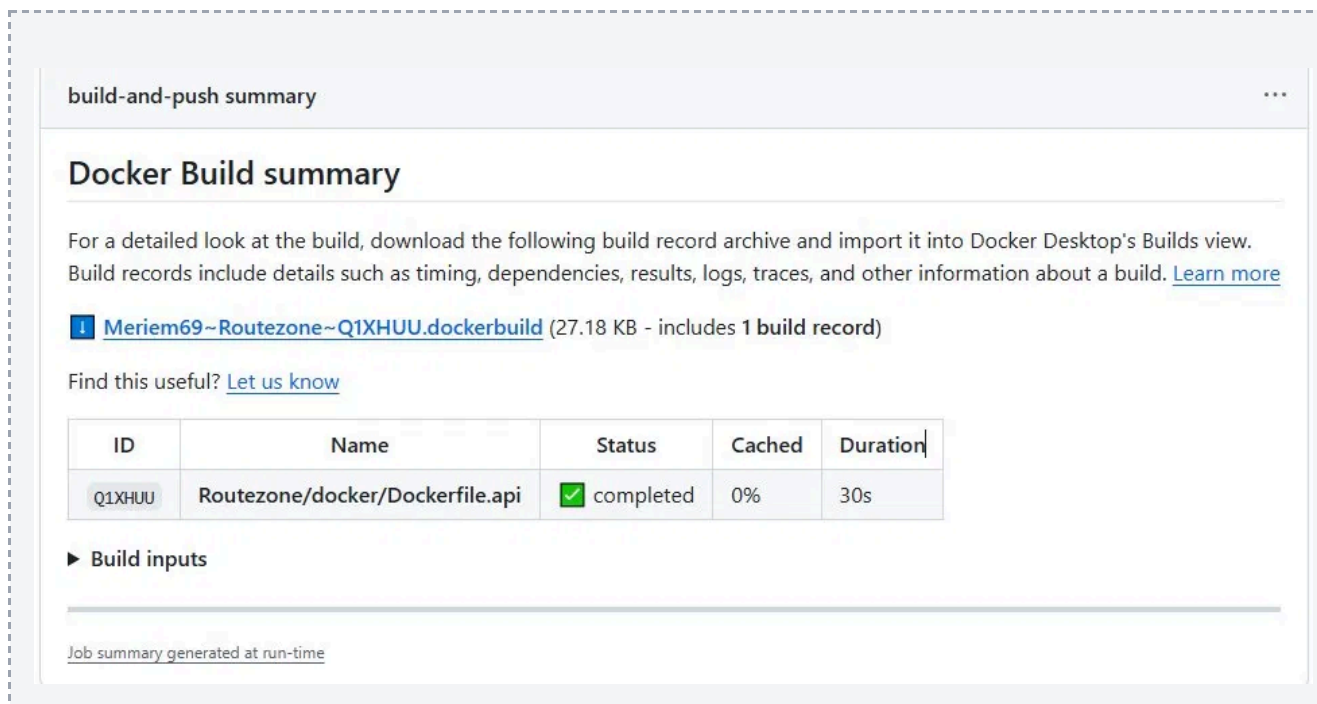


Figure 13 — Build de l'image terminé sur docker/Dockerfile.api

8.6 Scan de sécurité automatique avec Dependabot

Dependabot est activé sur mon repository GitHub. Cet outil scanne automatiquement mes dépendances Python (FastAPI, LightGBM, Pandas, etc.) pour détecter les bibliothèques contenant des failles de sécurité connues (CVE - Common Vulnerabilities and Exposures). Lorsqu'une vulnérabilité est détectée, Dependabot crée automatiquement une Pull Request proposant la mise à jour vers une version corrigée. C'est un filet de sécurité supplémentaire qui s'intègre naturellement dans ma démarche MLOps : sans surveiller toutes les CVE manuellement, je suis prévenue dès qu'une faille critique apparaît sur une de mes dépendances.

8.7 Distinction des chaînes et CD

2 chaînes coexistent dans le projet. La chaîne du modèle valide la qualité des prédictions : le test de non régression du Recall agit comme garde-fou avant toute livraison d'un modèle réentraîné. La chaîne de l'application automatise les tests du code à chaque push, puis le packaging : une fois les 35 tests verts, l'image docker est construite et publiée sur le registre Github (GHCR). Le packaging et la livraison de l'artefact sont donc automatisés; le déploiement continu sur un serveur distant (Render, Railway, VPS) reste la suite directe, déjà cadrée dans les perspectives.

8.8 Limites assumées

Pas de déploiement continu sur serveur distant : la chaîne build et push l'image Docker sur GHCR, mais son déploiement automatique sur un serveur en production (Render, Railway, VPS) reste la dernière étape, déjà cadrée dans les perspectives. La livraison de l'image, elle, est automatisée

9. Monitoring temps réel

9.1 Pourquoi monitorer

Même si tous mes tests sont verts, je dois surveiller l'application après son démarrage pour détecter rapidement les anomalies. Sans monitoring, je découvrirais les bugs quand un utilisateur viendrait se plaindre, trop tard.

Sur un projet ML, le monitoring sert aussi à détecter le drift du modèle : la dégradation progressive des prédictions dans le temps. Si les données réelles changent (nouveaux véhicules, nouvelles infrastructures), le modèle peut perdre en précision sans que ça apparaisse dans les métriques classiques.

9.2 Architecture du monitoring en 3 piliers

- Prometheus : collecte les métriques (compteurs, latences, erreurs) toutes les 15 secondes
- Grafana : affiche les métriques sous forme de dashboards temps réel
- Python Logging : enregistre chronologiquement les événements dans logs/api.log

9.3 Métriques Prometheus exposées

DÉFINITION — Prometheus système open source de monitoring orienté time-series. Interroge périodiquement les services (toutes les 15s par défaut) pour récupérer leurs métriques et les stocker avec horodatage.

4 métriques sont définies dans le module metrics.py :

- routezone_predictions_total (Counter) : nombre total de prédictions
- routezone_predictions_par_classe_total (Counter) : par classe Grave / Pas grave
- routezone_prediction_duration_seconds (Histogram) : latence des prédictions
- routezone_errors_total (Counter) : nombre total d'erreurs

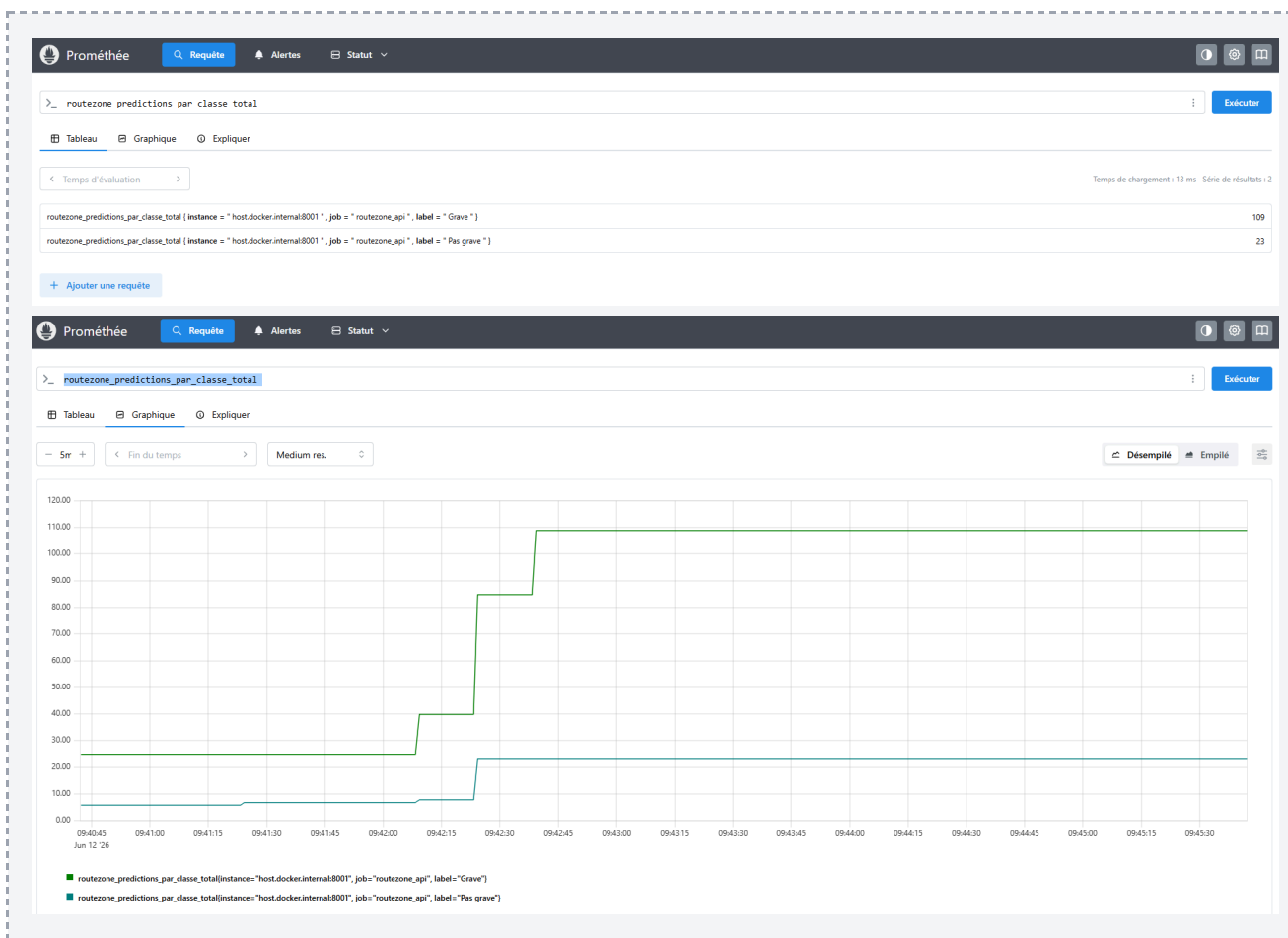


Figure 14— Prometheus : métrique routezone_predictions_par_classe_total

La métrique **routezone_predictions_par_classe_total** expose deux séries temporelles distinctes via le label label : une pour la classe Grave (109 prédictions) et une pour la classe Pas grave (23 prédictions). Cette ventilation permet de surveiller en temps réel la distribution des prédictions du modèle et de détecter rapidement un éventuel drift, c'est-à-dire une dégradation des prédictions par rapport à la distribution attendue.

9.4 Dashboard Grafana

DÉFINITION — Grafana plateforme open source de visualisation de métriques. Lit Prometheus en lecture seule, n'stocke rien lui-même. Rafraîchissement temps réel toutes les 5 à 15 secondes.

Mon dashboard contient 5 panels couvrant les indicateurs essentiels :

- Nombre total de prédictions (compteur cumulatif)
- Prédictions par classe (répartition Grave / Pas grave en temps réel)
- Latence moyenne sur 5 minutes (performance API)
- Prédictions par minute (charge du service)
- Total des erreurs (fiabilité)



Figure 15 — Dashboard Grafana RouteZone : monitoring temps réel

Le dashboard Grafana centralise les 5 indicateurs clés de monitoring : nombre total de prédictions (132), répartition par classe (Grave / Pas grave), latence moyenne (26,7 ms par prédiction), débit par minute, et taux d'erreurs. Toutes ces métriques sont alimentées par Prometheus en temps réel, avec un rafraîchissement toutes les 15 secondes.

9.5 Python Logging

Le module logger.py écrit dans la console et dans le fichier logs/api.log avec un format structuré : 2026-05-13 14:23:18 | INFO | routezone | message. Niveaux utilisés : INFO (événements normaux), WARNING et ERROR (anomalies). Le fichier est persistant : l'historique est conservé même après redémarrage de l'API.

9.6 Différence entre tests et monitoring

Les tests vérifient que le code fonctionne avant la mise en ligne. Test échoue, je corrige avant de déployer. Le monitoring surveille ce qui se passe après la mise en ligne, en conditions réelles. Anomalie détectée, je suis prévenue immédiatement. Aucun des deux ne remplace l'autre : les tests préviennent les bugs connus à l'avance, le monitoring détecte les problèmes imprévus à l'usage.

9.7 Limites assumées

- Pas d'alertes automatiques : aujourd'hui je dois regarder Grafana manuellement. Alertmanager + Slack/email à configurer
- Monitoring en local uniquement : en production, externaliser via Grafana Cloud ou Datadog
- Pas de monitoring du drift modèle : Evidently AI ou MLflow Model Monitoring à ajouter

10. MLflow tracking

10.1 Présentation rapide de MLflow

DÉFINITION — MLflow plateforme open source créée par Databricks pour gérer le cycle de vie d'un modèle ML. Quatre modules : Tracking (suivi d'expériences), Projects (packaging), Models (déploiement standardisé), Registry (gestion formelle des versions).

Dans RouteZone, j'utilise principalement MLflow Tracking. Détaillé dans le rapport E2 (encadré 3).

10.2 Ce qui est tracké à chaque expérimentation

- Hyperparamètres : max_depth, learning_rate, num_leaves, class_weight, etc.
- Métriques : Recall GRAVE, Precision GRAVE, F1 macro, AUC-ROC
- Artefacts : modèle .pkl, matrice de confusion, feature importance
- Métadonnées : nom du run, date, durée, source du notebook

10.3 L'interface MLflow UI

MLflow UI est accessible via la commande `mlflow ui` depuis le terminal (dans le dossier notebooks/), sur <http://localhost:5000>. L'interface permet de voir les runs côte à côte, filtrer par métrique, comparer plusieurs runs, télécharger le modèle entraîné.

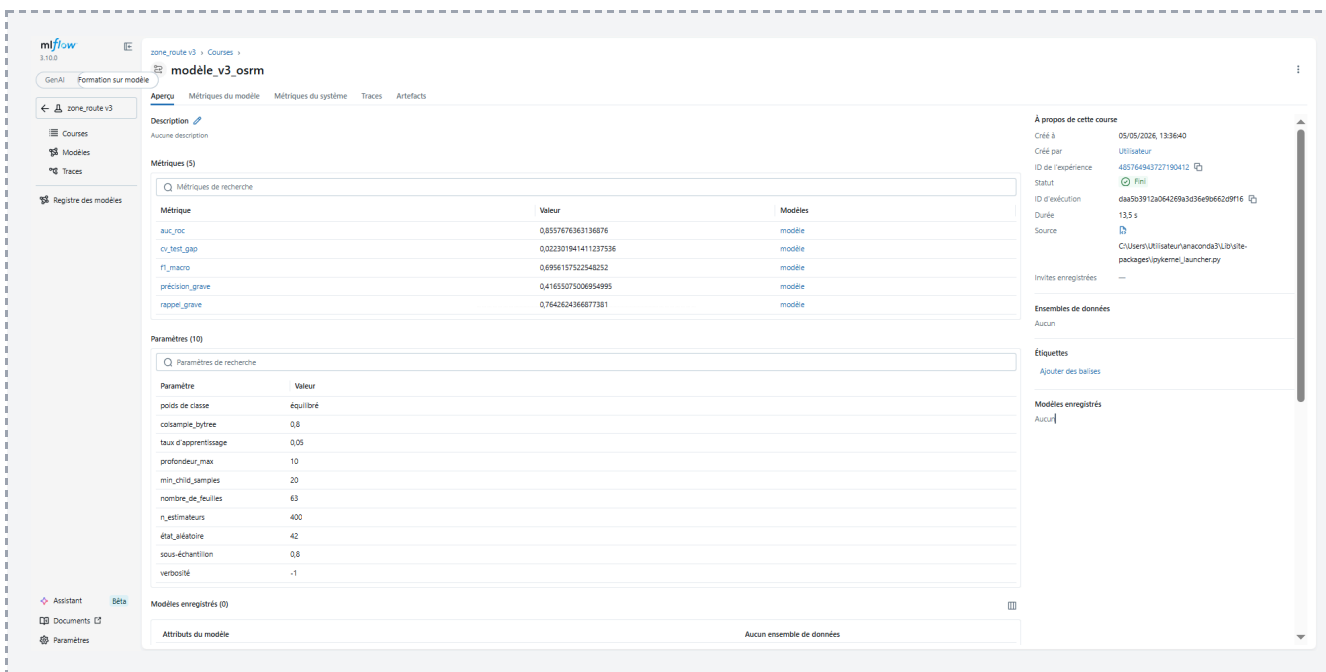


Figure 16 — Détail du run MLflow `modele_v3_osrm` : 5 métriques et 10 hyperparamètres trackés pour reproductibilité totale du modèle de production.

L'interface MLflow UI affiche le détail du run `modele_v3_osrm` correspondant au modèle de production. Cinq métriques sont enregistrées, dont l'AUC-ROC à 0.8558 et le Recall GRAVE à 0.7643. Dix hyperparamètres sont tracés, dont la profondeur maximale (10), le nombre d'estimateurs (400), le `learning_rate` (0.05) et le `class_weight` balanced. La métrique `cv_test_gap` à 0.022 confirme la robustesse du modèle : l'écart entre validation croisée et test est minimal, preuve d'une bonne généralisation.

10.4 Comment MLflow a influencé mes décisions

Sans MLflow, j'aurais perdu le fil de mes expérimentations. J'ai testé plus de 10 configurations différentes. MLflow m'a permis de :

- Comparer V1, V2, V3 OSRM côte à côte : c'est ce qui a justifié le choix de V3 (meilleur Recall GRAVE)
- Comparer 4 stratégies d'optimisation (tuning manuel, GridSearch, Optuna Recall, Optuna F1)
- Documenter objectivement l'incident calibrator : la chute du Recall à 33 % est tracée dans MLflow

10.5 Limites assumées

- Pas de MLflow Registry : aujourd'hui uniquement Tracking. Le Registry permettrait de gérer formellement les étapes Staging / Production / Archived
- MLflow local uniquement : en production, serveur centralisé accessible à l'équipe
- Pas de tracking continu en CI : intégrer MLflow dans la CI/CD pour un run automatique à chaque réentraînement

11. Conclusion

11.1 Récapitulatif

Ce rapport E3 a présenté la mise en service complète du modèle LightGBM RouteZone. L'architecture repose sur 5 containers Docker orchestrés par docker-compose : une API REST FastAPI unifiée (16 endpoints sécurisés par clé API et JWT), une base PostgreSQL pour les utilisateurs et l'historique, un moteur OSRM pour les temps d'intervention en temps réel, une application Streamlit pour le frontend, et une chaîne de monitoring complète (Prometheus + Grafana + Python Logging). L'application est testée par 35 tests pytest (dont 1 test de non-régression Recall), et chaque push sur GitHub déclenche une chaîne CI avec GitHub. Le cycle de vie du modèle est tracé avec MLflow.

11.2 Points forts

- Sécurité : 5 mesures OWASP adressées (A01, A02, A03, A07, A09)
- Conformité RGPD : 2 endpoints dédiés (articles 17 et 20)
- Cohérence métier validée : 5 tests spécifiques (moto > voiture, frontale > arrière, etc.)
- Démarche scientifique : décisions documentées et chiffrées (calibrator désactivé, choix V3 OSRM)
- Reproductibilité totale : projet entièrement portable via Docker
- Industrialisation MLOps complète : tracking MLflow + tests pytest + CI/CD + monitoring temps réel

11.3 Limites assumées

Toutes les limites suivantes sont identifiées, documentées, et un plan d'amélioration est défini :

- Pas de déploiement automatique sur serveur : le packaging et la publication de l'image sont en place, seul le déploiement reste à ajouter
- Pas de HTTPS en local : reverse proxy nginx + SSL en production
- Pas de rate limiting : intégration de slowapi à prévoir
- Pas de tests automatisés sur Streamlit : tests manuels uniquement aujourd'hui
- Pas de couverture pytest-cov mesurée : amélioration prévue avec objectif 70-80 %
- Pas d'alertes Grafana : Alertmanager + Slack/email à configurer
- Pas de monitoring du drift modèle : Evidently AI à intégrer
- Pas de MLflow Registry : aujourd'hui uniquement Tracking
- Pas de fonction mot de passe oublié : email + lien de réinitialisation à ajouter

11.4 Ce que ce projet m'a apporté

RouteZone m'a fait passer du statut d'apprenante en ML à celui de Développeuse en Intelligence Artificielle capable de livrer un service de bout en bout. Trois enseignements majeurs :

D'abord, l'importance des fondations. Un modèle .pkl sur un poste de développement ne sert à personne. C'est l'ensemble du dispositif — API documentée, sécurité OWASP, conformité RGPD, monitoring temps réel, tests automatisés, CI/CD, conteneurisation — qui transforme un POC en outil potentiellement utile. J'ai appris à raisonner en termes de système, pas seulement de modèle.

Ensuite, la valeur des décisions assumées. Désactiver le calibrator isotonique alors qu'il améliorait l'Accuracy était contre-intuitif. Mais le Recall GRAVE chutait de 76 % à 33 %. Sur un système d'aide aux secours, c'est inacceptable. J'ai appris à choisir mes métriques en fonction du contexte métier, et à documenter les décisions techniques au lieu de les subir.

Enfin, la rigueur méthodologique. Chaque choix dans ce projet est tracé : 21 runs GitHub Actions, 35 tests pytest, plusieurs runs MLflow, un dashboard Grafana avec 5 panels, des logs structurés. Si demain un bug apparaît en production, je sais où chercher. Cette traçabilité, ce n'est pas un luxe, c'est ce qui distingue un POC robuste d'un prototype jetable.

11.5 Perspectives concrètes

Court terme (V1.1)

- Intégrer slowapi pour le rate limiting sur /auth/login
- Mesurer la couverture des tests avec pytest-cov (objectif 70-80 %)
- Configurer Alertmanager + email pour les pics d'erreurs Grafana

Moyen terme (V2)

- Ajouter le déploiement continu sur serveur (le packaging et la publication d'image sont déjà en place)
- Mettre en place MLflow Registry pour gérer formellement Staging / Production / Archived
- Intégrer Evidently AI pour le monitoring du drift modèle
- Ajouter une fonction mot de passe oublié (email + lien de réinitialisation)

Long terme (V3)

- Déploiement sur VPS ou cloud (Scaleway, OVH) avec HTTPS et reverse proxy
- Application mobile native pour les forces de l'ordre et services de secours
- Connexion temps réel aux systèmes d'information des SAMU et SDIS
- Étude d'impact réel : mesurer le gain en temps de prise en charge sur un terrain test